UTN FRLP

Desarrollo de Software 2026

Deploy y Entornos

Clase #23 - 19 de octubre de 2026

Matías Corti - Martina García Améndola



La deuda que traemos

Lo que dejamos pendiente en clases anteriores y hoy se resuelve:

Pendiente	**Dónde quedó**	**Hoy**
HTTPS, dominio real, CORS de producción	C19	Render da HTTPS; FRONTEND_URL deja de ser localhost
"Anda en mi máquina" → tiene que andar en todas	C13	Una imagen de producción corre en una compu ajena
CI: que los tests corran solos en cada push	C21	Vemos CD (auto-deploy). CI queda nombrado, no hecho
Fechas de presentación: orden definitivo	C21	Calendario y orden, al cierre

Temas de hoy

1. **Qué significa** poner la app en producción
2. **Qué cambia** respecto de correrla en tu compu: entornos, variables, build, imagen
3. **Cómo lo resolvemos** en la materia y por qué: Render + Neon
4. **Cómo lo hacen ustedes:** guía para deploy
5. **Cómo se mantiene solo:** auto-deploy al pushear
6. **Qué viene:** Entrega 4, presentaciones, cierre



¿Qué es deployar? Lo mismo, en otra compu

	Tu compu (desarrollo)	**Una compu ajena (producción)**
Acceso	http://localhost:5173	https://libreria.onrender.com
Quién la prende	vos, docker compose up	la plataforma, sola
Cuándo corre	mientras la usás	24/7 (o hasta que se duerme)
Cómo entran los cambios	guardás el archivo	pusheás a main
Errores	se toleran	los ve la gente
Datos	tu Postgres de Docker	una DB en internet

- **Lo que NO cambia:** el código.
- **Lo que cambia:** dónde corre, quién lo arranca, con qué variables, cómo se construye.
- Deploy = deployment = despliegue. Es lo mismo.



Entornos: dev staging prod

Entorno	Qué es	Quién lo usa	En la materia
Development	tu máquina: rompés, arreglás, experimentás	cada dev	Docker en tu compu
Staging	copia "fiel" de producción para probar antes de liberar	QA, el cliente	no (fuera de alcance)
Production	lo que usan los usuarios: estable, monitoreada	el mundo	Render + Neon

- Un entorno no es un lugar: es **dónde corre + qué variables tiene**.
- Mismo código en los tres. Lo que cambia es la configuración.
- Un equipo puede tener más (QA, demo, preprod): el concepto es el mismo.



Variables por entorno: de .env al panel

Las mismas variables que ya usan, con otro valor y en otro lugar:

Variable	Dev (backend/.env)	Prod (panel de Render)	Quién pone el valor
PORT	3000	la inyecta Render	Render
FRONTEND_URL	http://localhost:5173	https://{{front}}.onrender.com	vos
DATABASE_URL	postgresql://…@db:5432/libreria_db	postgresql://…@ep-….neon.tech/neondb?sslmode=require	Neon
JWT_SECRET	una de mentira	generada por Render (botón)	Render
VITE_API_URL (front)	http://localhost:3000/api	https://{{back}}.onrender.com/api	vos, en el build

- **No hay `.env.production` en el repo.** La configuración de producción vive en la plataforma, no en Git.
- `.env.example` documenta **qué** variables existen. El **valor** lo pone cada entorno.
- El código no cambia: `process.env.PORT`, `process.env.FRONTEND_URL`, `import.meta.env.VITE_API_URL`.



Secretos: lo que el front no puede guardar

```
cd frontend && npm run build
grep -o "onrender.com/api" dist/assets/index-*.js
# → https://libreria-api.onrender.com/api   ← literal, en un archivo
#   que baja cualquiera
```

	Backend	**Frontend**
Dónde corre	en el servidor	en el navegador del usuario
Cuándo lee las variables	al arrancar	en el build (quedan pegadas al JS)
Quién las ve	sólo el servidor	cualquiera, con F12
Qué puede ir	JWT_SECRET, DATABASE_URL	URLs públicas, flags, IDs públicos

- Regla: **si te importa que nadie lo lea, no va al front.** VITE_ = público.
- JWT_SECRET en producción se **genera** en Render. Nadie lo escribe, nadie lo pega en Discord.

✅

Mini-cierre A

Concepto	En una línea
Deployar	El mismo código, en una compu ajena, con otras variables
Entorno	Dónde corre + qué variables tiene
dev / staging / prod	Tu compu / copia para probar / lo que usa la gente
.env vs panel	Local, en archivo que no va a Git; producción, en la plataforma
.env.example	Qué variables hay, sin valores
Secreto	Sólo en el back. En el front todo queda en el bundle



Dev server vs build: el front son archivos

	npm run dev	**npm run build**
Qué corre	Vite, un servidor en Node	Nada: genera archivos y termina
Salida	nada en disco	dist/
Tamaño	muchos archivos, sin comprimir	pocos, minificados, con hash
Variables	se leen al vuelo	quedan embebidas
Quién lo sirve	Vite, en tu compu	cualquier servidor de archivos

```
dist/
├── index.html
└── assets/
    ├── index-BQkeVOpy.js    382 kB → 119 kB gzip
    └── index-AbXnwv5E.css
```

- En producción el front **no tiene Node**. Son tres archivos y un servidor que los entrega.
- Por eso en Render es un **Static Site**: build → publicar `dist/`.



404 AL REFRESCAR: RUTAS DE UNA SPA

1. Entrás a `/` → el servidor manda `index.html` ✅
2. Click a `/catalogo` → React Router cambia la URL **sin pedirle nada al servidor** ✅
3. F5 en `/catalogo` → el servidor busca un archivo `/catalogo` → no existe → **404** ❌

La solución es una regla en el Static Site:

Source	Destination	Action
/*	/index.html	Rewrite

- El servidor devuelve **siempre** `index.html`. React Router mira la URL y decide qué mostrar.
- Rewrite ≠ redirect: la URL del navegador no cambia.



El back también se compila

	Dev	Prod
Comando	npm run dev = tsx watch src/index.ts	npm run build = tsc -p tsconfig.build.json, después node dist/index.js
TypeScript	se transpila en memoria, cada vez	una vez, en el build
Reinicio al guardar	sí	no hay "guardar"
Tests	vitest los encuentra en src/	no van a dist/

```
// backend/tsconfig.build.json
{
  "extends": "./tsconfig.json",
  "compilerOptions": { "outDir": "dist", "rootDir": "src", "sourceMap": false },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist", "src/**/*.test.ts"]
}
```

- `tsx` es para desarrollar. En producción corre Node con JavaScript ya compilado.
- Nuevos en `package.json`: `"build"` y `"start": "node dist/index.js"`.



Imagen de desarrollo vs imagen de producción

	Dockerfile (dev, desde C13)	**Dockerfile.prod (hoy)**
Para qué	programar	correr
Código	montado por volumen: cambia en vivo	copiado adentro: fijo
Qué instala	todo, y queda	todo para compilar; la imagen final copia lo justo
CMD	npm run dev	prisma migrate deploy && node dist/index.js
Migraciones	las corrés vos (migrate dev)	las corre la imagen al arrancar
Quién la buildea	docker compose up --build	Render, en cada push

- Dos archivos porque son **dos trabajos**. El `docker-compose.yml` sigue usando el de dev, sin cambios.
- La imagen de producción no tiene `src/`, ni `tsx`, ni tests: sólo `dist/` y lo que Prisma necesita para migrar.



Dockerfile.prod: dos etapas

```dockerfile
FROM node:22-alpine AS build          # etapa 1: compilar
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate && npm run build

FROM node:22-alpine                   # etapa 2: correr
WORKDIR /app
ENV NODE ENV=production
COPY --from=build /app/node modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package*.json prisma.config.ts ./
COPY prisma ./prisma
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/index.js]'
```

- **Multi-stage:** la etapa 1 tiene todo lo necesario para compilar; la 2 se queda sólo con lo que corre.
- `COPY --from=build`: `node_modules` se instala **una vez**, la etapa 2 lo copia.
- Sin `ARG` ni `ENV` con secretos: `DATABASE_URL` y `JWT_SECRET` entran al **correr**, no al construir.



Migraciones al arrancar

	prisma migrate dev	prisma migrate deploy
Dónde	tu compu	producción
Qué hace	crea la migración nueva y la aplica	aplica las que ya están en prisma/migrations/
Genera SQL	sí	no: sólo ejecuta el que está commiteado
Si algo no cuadra	pregunta, puede resetear	falla

```
npx prisma migrate deploy && node dist/index.js
```

- Por eso las migraciones van **commiteadas**: en producción no se inventan, se aplican.
- Si una migración falla → el `CMD` falla → el deploy falla → **sigue corriendo la versión anterior**.
- Prisma: la misma versión que el repo. Nada de `npm i prisma@latest` "para deployar".



Mini-cierre B

Concepto	En una línea
Build	De código fuente a archivos que corren solos
Front en prod	Archivos estáticos, sin Node, variables embebidas
Rewrite SPA	/* → /index.html; React Router hace el resto
Back en prod	tsc → node dist/index.js. Sin tsx, sin watch
Imagen dev vs prod	Programar vs correr: dos Dockerfiles
Multi-stage	Etapa 1 compila; etapa 2 se queda sólo con lo necesario
migrate deploy	Aplica lo commiteado, no inventa. Falla = no se deploya

Arquitectura de producción de la Librería



👤 **Usuario**
Navegador

https → archivos

🌐 **Frontend Estático**
Render > Static Site
fetch → VITE_API_URL

> frontend/ > publica dist/

Variable embebida en el build

API calls

🖥️ **Backend dinámico**
Render > Web Service
FRONTEND_URL → CORS
DATABASE_URL → Prisma

> backend/ > Dockerfile.prod

CORS quién puede llamarme
DB dónde están los datos

SQL queries

🗄️ **Base de datos**
Neon – PostgreSQL

Misma región que Render
(AWS us-east-1)

- Tres servicios, dos cuentas (Render y Neon), un repo.



Modelos de hosting: qué hace un PaaS por vos

Modelo	Vos manejás	Ejemplos	Complejidad
IaaS	la máquina: SO, Node, Postgres, parches	AWS EC2, Azure VM	alta
PaaS ⭐	el código; la plataforma pone máquina, HTTPS, logs	Render, Railway, Fly.io, Heroku	media
Serverless	funciones que corren sólo ante un request	AWS Lambda, Vercel Functions	baja, pero otro modelo

Lo que un PaaS resuelve solo: build desde Git, HTTPS, variables de entorno, logs, reinicios.



Por qué Render + Neon, y qué se usa afuera

Lo que pedimos para la materia: **gratis – sin tarjeta – Docker – Postgres que no venza – una cuenta por grupo.**

Alternativa	**Por qué no, en una línea**
Vercel (front)	otra cuenta más; un Static Site de Render enseña lo mismo
Render Postgres	la base gratuita vence a los 30 días
Supabase	conexión directa sólo por IPv6; hay que pasar por un pooler
Railway · Koyeb · Fly.io · DigitalOcean	piden tarjeta o no tienen capa gratuita
AWS · GCP · Azure	tarjeta y una consola enorme para una demo
GitHub Pages	sólo estático; queda como actividad individual opcional

- **En un trabajo se elige por:** costo, escala, equipo (qué sabe operar), cumplimiento (dónde pueden vivir los datos).
- La capa gratuita cobra en paciencia: el back **se duerme** sin tráfico y **tarda unos segundos en despertar**. El front no se duerme.

🔁

Auto-deploy: push → build → deploy

```
git push origin main
  → Render detecta el commit
  → ¿cambió frontend/?  → build del Static Site → publica dist/
  → ¿cambió backend/?   → build de la imagen → migrate deploy → arranca
  → si algo falla       → queda la versión anterior
```

	CI – integración continua	**CD – entrega continua**
Qué hace	corre tests y lint en cada push	publica automáticamente cada push a main
Responde	¿esto anda?	¿esto está online?
Hoy	no (GitHub Actions, fuera de alcance)	sí: Render

- Cambiar una variable de entorno **también** redeploya.
- Deployar sólo desde main. Cada push gasta minutos de build del plan gratuito.

✅ Mini-cierre C

Concepto	En una línea
Arquitectura	Static Site + Web Service Docker + Neon; cada flecha, una variable
PaaS	Vos el código; la plataforma, el resto
Elegir plataforma	Para la materia: gratis y simple. Afuera: costo, escala, equipo, cumplimiento
Auto-deploy	push a main → build → deploy. Falla = versión anterior
CD vs CI	CD publica, CI verifica. Hoy sólo CD
Capa gratuita	El back duerme: warm-up antes de la demo



Práctica

Orden de creación: el mapa de deploy (1 de 2)

#	Dónde	Qué	Me llevo
1	Neon	proyecto nuevo, región AWS us-east-1	DATABASE_URL (directa, no pooled)
2	Render	Web Service · Docker · root backend · Dockerfile.prod · env: DATABASE_URL, JWT_SECRET (generado), FRONTEND_URL=http://localhost:5173 provisorio	URL del back
3	Render	Static Site · root frontend · build npm run build · publica dist · env VITE_API_URL={{back}}/api · rewrite /* → /index.html	URL del front

- **Por qué este orden:** cada URL existe recién cuando el servicio existe. El back necesita la del front (CORS) y el front la del back (fetch): uno arranca con un valor provisorio.

Orden de creación: el mapa de Deploy (2 de 2)

#	Dónde	Qué	Me llevo
4	Render	back → FRONTEND_URL={{front}} (cambiar la variable redeploya)	-
5	Tu compu	docker compose run --rm -e DATABASE_URL="{{NEON}}" api npx prisma db seed	datos de demo
6	Navegador	abrir el front, loguearse, crear un libro, F5 en /catalogo	las dos URLs → Discord

- Las pantallas de cada paso están en la Guía de Deploy.



Checklist: repo listo para deploy

Backend

✅ `Dockerfile.prod` y `tsconfig.build.json` (base: branch `C23`)

✅ `npm run build` (`tsc -p tsconfig.build.json`) limpio

✅ `PORT` y `FRONTEND_URL` desde `process.env`, con default local

✅ `prisma/migrations/` commiteado y sin pendientes (`migrate dev` no pide nada)

✅ `.env.example` con todas las variables, sin valores reales

✅ `src/generated` en `.gitignore`

Frontend

✅ `npm run build` limpio, con los tests adentro

✅ la API se alcanza **sólo** por `VITE_API_URL`: ningún `localhost:3000` suelto en `src/`

✅ rutas protegidas funcionando (`/libros/nuevo` sin login → `/login`)

✅ `.env.example` con `VITE_API_URL`

Repo: sin `.env`, sin `dist/`, sin `node_modules/` commiteados. Todo en `main`.



Qué falla y por qué (1 de 2)

Síntoma	Causa	Arreglo
Consola: fetch a undefined/libros	VITE_API_URL no estaba al hacer el build (compila igual, sin avisar)	setear la variable en el Static Site y redeployar
404 en todos los pedidos a la API	VITE_API_URL sin /api al final	https://{{back}}.onrender.com/api
Consola: bloqueado por CORS	FRONTEND_URL sigue en localhost o tiene / al final	la URL exacta del front, sin barra
Log: Falta JWT_SECRET en el .env	la variable no está en el panel	agregarla (botón generar)
Log: The datasource.url property is required	falta DATABASE_URL	agregarla



Qué falla y por qué (2 de 2)

Síntoma	Causa	Arreglo
Log: P1001 Can't reach database server	URL mal pegada, o pooled en vez de directa	copiarla de nuevo desde Neon
404 al refrescar en /catalogo	falta la regla de rewrite	/* → /index.html
Build falla con error TS…	el tsc que "casi" andaba	arreglarlo local primero: el build no perdona
Log: SECURITY WARNING: The SSL modes…	aviso informativo de pg sobre sslmode	no es error; seguir
Primer request tarda ~1 minuto	back dormido	no es error; warm-up

La diferencia entre un deploy y un dolor de cabeza: **leer el log.**



Lo que viene: calendario y orden de presentación

Fecha	Qué	Grupos, en este orden
26/10 · C24	Presentaciones, tanda 1	G07 · G05 · G09
02/11 · C25	Presentaciones, tanda 2	G02 · G10 · G08 · G11
09/11	Feriado	–
15/11	**Entrega Final TP**	**todos**
16/11 · C26	Presentaciones, tanda 3	G04 · G06 · G01 · G03
23/11	Feriado	–
30/11 · C27	**Recuperatorio del examen**	**quienes recuperan**

- No hay más clases de contenido. Las consultas siguen en Discord y en los días de presentación.
- Cada grupo presenta la app local, **la app deployada debe estar para la Entrega Final.**



Entrega 4 (15/11): checklist

Requerido

✅ App deployada: URL del front y del back funcionando (Render + Neon)

✅ Tests: Vitest en back y front, `npm test` verde en los dos

✅ `README.md`: cómo correrla local (Docker) **y** cómo está deployada: servicios, variables (sin valores), cómo migrar y cargar datos

✅ `Documentación/` al día: matriz de permisos, endpoints, backlog cerrado en Trello

✅ Usuarios de prueba para que la probemos: en el canal del grupo, no en el README

✅ Participación de todos: PRs con review cruzado

Fundamental: avisar en el canal del grupo cuando esté subido.



Fuera de alcance

Tema	Qué es	Por qué no hoy
render.yaml	infraestructura como código	la interfaz alcanza para tres servicios
Dominio propio	libreria.com.ar en vez de .onrender.com	cuesta plata y no cambia nada técnico
Staging	un entorno intermedio	un tercer deploy por grupo; el concepto quedó en la slide 5
Monitoreo y alertas	enterarse de que se cayó antes que el usuario	otra materia
Pooler de Neon	multiplexar conexiones	con el adapter y un solo back no hace falta
Front dockerizado para dev	el front adentro de docker compose	Vite en el host anda y es más rápido
GitHub Actions (CI)	tests en cada push, antes del deploy	así se agregaría CI; lo nuestro es CD



Deuda técnica: lo que un equipo real hace distinto

Deuda	Lo que hacemos	Lo que hace un equipo real
Migraciones	corren en el CMD, al arrancar	paso aparte del pipeline, antes de levantar la versión nueva
Datos iniciales	seed a mano desde la compu de alguien	script versionado, o staging con datos propios
Token	localStorage (deuda desde C20)	cookie httpOnly + refresh token
CI	no hay	tests y lint en cada PR; el deploy no sale si fallan
Secretos	se pegan en un panel	gestor de secretos, rotación periódica
Back dormido	arranque en frío	instancia paga, o healthcheck que lo mantiene despierto

Cuándo se aplica: en el trabajo. Lo importante es saber que está.



La presentación



La presentación: formato y esquema

⏱️ **20 minutos** máximo + **3 a 5** de preguntas individuales, todos participan, **3 a 4 minutos** cada uno.

Como si vendieran el producto, no como si rindieran.

Esquema sugerido

1. **Introducción (2-3'):** equipo, problema que resuelven, usuario, propuesta de valor
2. **Demo (~10')** ⭐ - la app deployada, los flujos principales, como lo usaría un usuario real
3. **Experiencia y aprendizajes (3-5')** - desafíos, decisiones técnicas, trabajo en equipo
4. **Cierre (2-3')** - qué falta, qué harían distinto



Checklist pre-presentación

Requerido

✅ Presentación preparada (PDF/PPT), en su notebook

✅ Notebook preparada para conectar y proyectar (con todos los recursos instalados localmente), con todo abierto

✅ App verificada **ese mismo día**: las dos URLs responden

✅ Orden de participación definido

✅ Presentación completa y flujos de la demo ensayados al menos una vez, con cronómetro

Recomendado

✅ Pestañas abiertas con las URLs; datos de demo cargados

✅ Por las dudas llevar todo listo en más de una notebook



Tips

✅ Hablar natural, no leer un guión

✅ Mostrar la app como la usaría un usuario real

✅ Si algo falla, manejarlo con profesionalismo: seguir

✅ Respetar el tiempo

❌ Jerga técnica excesiva

❌ Desbalance en la participación

❌ Demo sin probar antes



Cierre de la materia

En marzo, para la mayoría, un era una novedad.

HTML y CSS → TypeScript → React con Zod → Docker → Express, Prisma y JWT → Vitest → **hoy: en producción, con tests, en equipo, con un cliente real.**

Es su primer proyecto completo. En las próximas semanas lo muestran.

¡Disfrútenlo! Y estén orgullosos de haberlo construido.

Actividades

Actividad Individual

[AI-C23] Deploy en la Librería

Referencia en el repo de ejemplo branch C23

1. Deployar el trabajo individual (Back y Front) en Render y Neon o alguna otra PaaS, siguiendo la Guía para Deploy: Neon → back → front → CORS → seed
2. **Opcionalmente**, para conocer una segunda plataforma, pueden deployar el front en GitHub Pages
3. Avisá en #actividades
   → hilo **[AI-C23] Deploy en la Librería**

Instancia final **15/11/2026**

⚠️ ***Cierre Bloque 4*** ⚠️

También es el límite para todas las actividades previas del año

Actividad Grupal

Entrega: **15/11/2026**

📌 **RECORDATORIO**
El **15/11/2026** es la **ENTREGA 4: Entrega Final** del **Trabajo Práctico Grupal**

[AG-C23] Deploy del TP Grupal

1. Repo listo para deploy (checklist de hoy), todo en main
2. Cuentas de Render y Neon
3. Deploy siguiendo la Guía para Deploy: Neon → back → front → CORS → seed
4. Probar en la URL pública: login, un CRUD completo, F5 en una ruta interna
5. README.md: sección "Deploy" (servicios, variables sin valores, cómo migrar y cargar datos), dejar las dos URLs
6. Enviar usuarios de prueba registrados por el Canal del Grupo de Discord (no en el repo).
7. Aviso en #actividades
   → hilo **[AG-C23] Deploy del TP Grupal**